Convolutional networks

10-701 Introduction to Machine Learning

Geoff Gordon

Signals as tensors

- An n-dimensional signal is represented as a tensor with n indices
  - e.g., grayscale image: 2 indices (pixel i, j)
  - e.g., color image: 3 indices (last is channel: R, G, or B)
  - e.g., pressure in a volume: 3 indices (voxel i, j, k)
- Each index also called a ***mode***
  - helps with overloading of the word “dimension”
- List of index ranges: the ***shape*** of the tensor
  - e.g., RGB image at VGA resolution: (640, 480, 3)
  - shape (3, 4, 2, 5) tensor has 120 total elements
  - first index can be 0, 1, 2; second 0, 1, 2, 3; …

Tensor operations

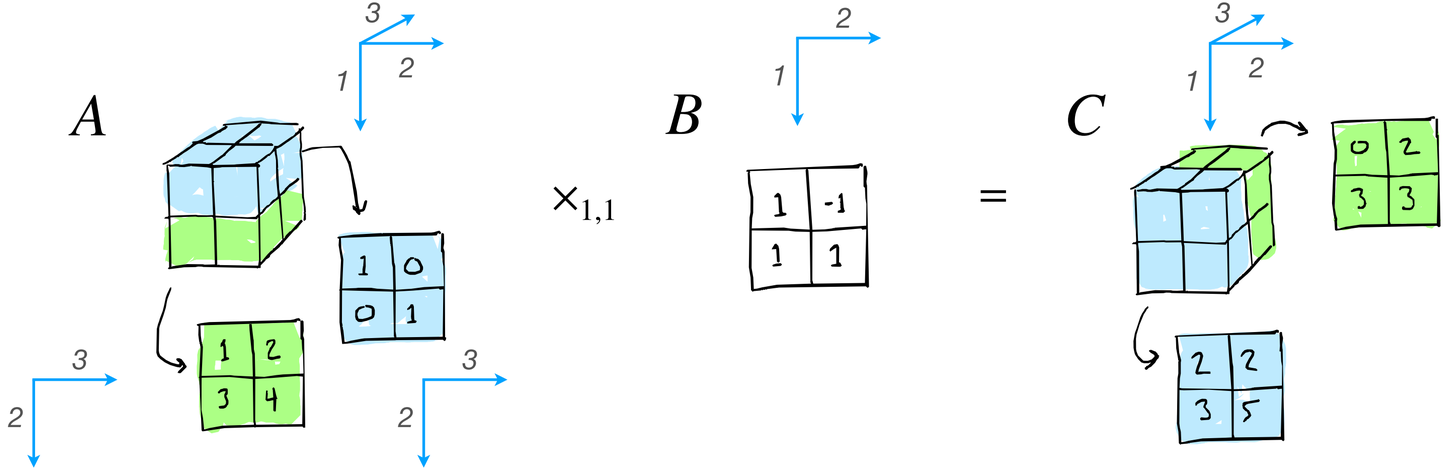
- Componentwise $+$, $\times$, $\cos$, …
  - with *broadcasting:*

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} + \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 2 & 3 & 4 \\ 6 & 7 & 8 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \times \begin{pmatrix} 1 & 2 & 3 \end{pmatrix} = \begin{pmatrix} 1 & 4 & 9 \\ 4 & 10 & 18 \end{pmatrix}$$

- Implemented as NumPy `Array` or PyTorch `Tensor`

Tensor operations



- Contraction: e.g., $A \times_{3,2} B$ or $A \times_{1,1} B$
  - like matrix multiplication (precise definition below), but using mode 3 of A and mode 2 of B — regular matrix multiplication is $\times_{2,1}$
  - modes of $A \times_{3,2} B$: modes of A (except 3) then modes of B (except 2)
  - by convention, all of A's modes first (in order) then B's
- Represents a linear function of B with parameters A
  - or a linear function of A with parameters B

Elementwise notation

- (a_{ij}) or a_{ij} means a matrix whose i, j element is a_{ij}
  - range of i, j can be left implicit or specified separately: e.g., $i \in \{1 \ldots 5\}, j \in \{1 \ldots 7\}$
- Similarly, a_{ijkl} means a 4-mode tensor
- So does $a_{ijk} b_{jkl}$ — the i, j, k, l element is $a_{ijk}\, b_{jkl}$
- So does $a_{ijk} + b_{jkl}$ (with $+$ instead of $\times$)
- The expression tells us how to compute a generic element of the result tensor from elements of the input tensors

Einstein summation

- We allow indices to appear in either raised or lowered positions: both a_i^j and a_j^i refer to 2-mode tensors (matrices) whose indices are i and j
- Only effect of raising and lowering is the ***Einstein summation convention****:* any index that appears ***both*** raised and lowered in an expression gets summed out
  - in $a_{ij}^k b_{jk}^l c_{lm}$ indices i, j, m appear only lowered; k, l are both raised and lowered
  - so the result is a tensor f_{ijm}: element at index i, j, m is $\sum_k \sum_l a_{ijk} b_{jkl} c_{lm}$
- Remaining indices keep their positions (raised or lowered)

Example

- If

$$a_i^j = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \qquad b_j = \begin{pmatrix} 1 \\ -1 \end{pmatrix} \qquad c^i = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

- then

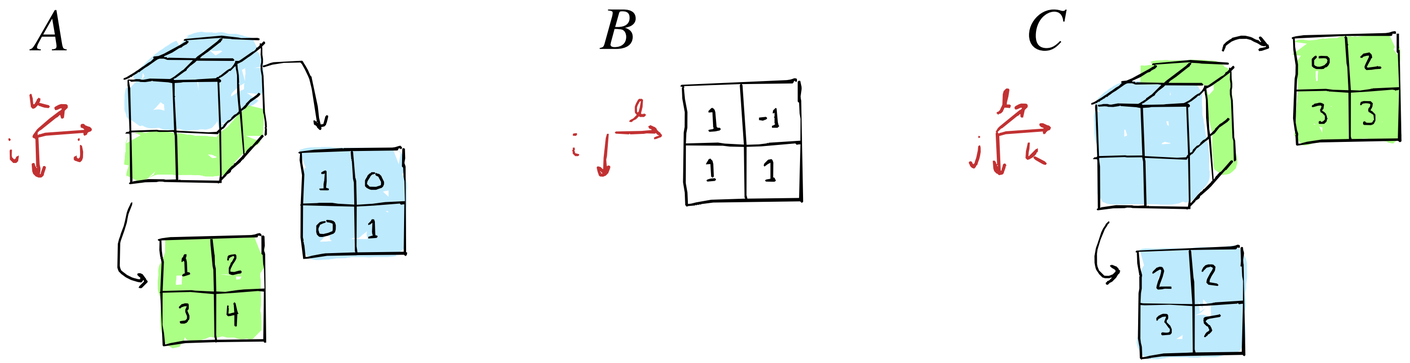
$$a_i^j b_j = \begin{pmatrix} -1 \\ -1 \end{pmatrix}$$

$$a_i^j c^i = \begin{pmatrix} 4 & 6 \end{pmatrix}$$

Transpose

- Changing around raised and lowered indices is a generalization of transpose
  - a^i_j and a^j_i have the same elements, as do x_j and x^j
  - convention: lowered indices in a 1- or 2-mode tensor correspond to rows, raised ones to columns
  - so x_j is a column vector (index runs through rows)
- With this convention:
  - Ax is $a^j_i x_j$
  - and $A^\top y$ is $a^i_j y_i$ (i acts like column index when we raise it, and j acts like row index when we lower it; since the elements are the same, a^i_j represents $A^\top$)
  - and $y^\top A$ is $a^j_i y^i$ (raised index of y, leaving a^j_i alone)

Tensor contraction as Einstein summation



$$A_{ijk}B^i_l = C_{jkl}$$

- To get contraction, raise exactly one index instance
  - e.g., $A_{ijk}B^i_l$ is a 3-mode tensor C_{jkl}
  - result of contracting A and B along the i index
  - element j, k, l is $\sum_i A_{ijk}B^i_l$

Python einsum

available in both NumPy and PyTorch, but some edge cases have different behavior

- A version of Einstein summation:
  - `np.einsum('ij,jk->ik', a, b)` is matrix multiplication
- Subscript string (first argument) formatted as `A,B->C`
  - part `A` names the indices of first input tensor `a`
  - part `B` names the indices of second input tensor `b`
  - part `C` tells us indices of output tensor
  - indices that don't appear in output are summed out
- Variants
  - can have any number of input tensors
  - `->C` is optional (defaults to all non-duplicated indices in alphabetical order)
  - broadcasting: `np.einsum('...ij->...ji', a)` transposes last two modes

Linear functions in derivatives

- Revisiting convention from earlier: if we have two tensors y_{ij} and x_{klm} then derivative $\frac{dy}{dx}$ is a 5-mode tensor representing a linear function from dx to dy
- We can write this linear function as a tensor contraction or Einstein summation:

$$dy_{ij} = A_{ij}^{klm}\, dx_{klm}$$

  - where A is the 5-mode tensor of coefficients
- Indices are $ijklm$, with last 3 indices (the ones corresponding to input x) raised
- Ex: Jacobian is $dy_i = A_i^j\, dx_j$

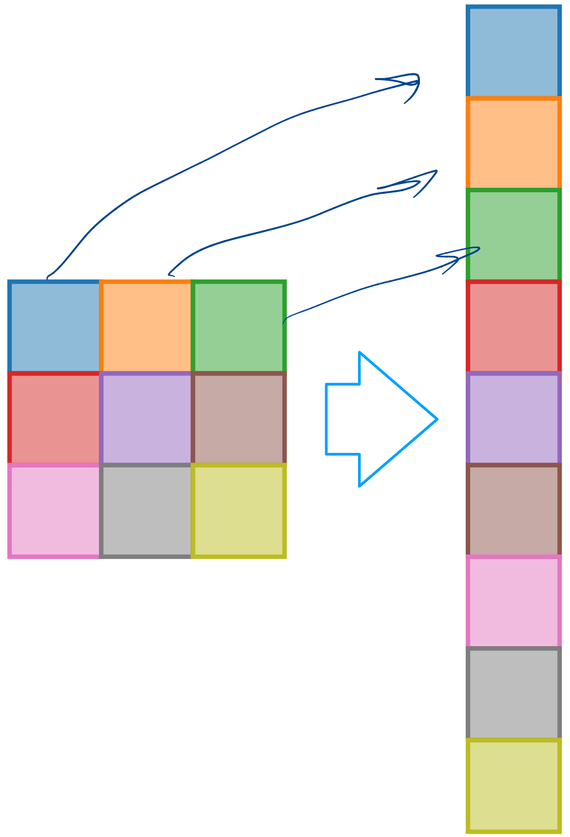
Batching

- Tensors make it easier to work with minibatches
- Just add a mode (typically last) that indexes samples within a batch
  - 640×480 images in batches of 17 $\rightarrow$ $640 \times 480 \times 17$
- Works with Python indexing conventions: if we drop the last index, we work with all samples at once

Convolutional neural networks

- We can use a convolution in a deep net layer
  - linear map $a \to a * b + c$ ($a, b \in \mathbb{R}^d$, $c \in \mathbb{R}$)
  - input a, weights b, bias c
  - learn b, c by SGD
- Variations: $b \in \mathbb{R}^k$ with $k \ll d$; omit c
- Nets with convolutions are *CNNs* or *convnets*
  - can also use the usual set of other blocks: fully connected linear, normalize, residual, pointwise nonlinearity, …

Connecting to MLP layers

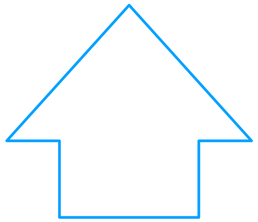


- To connect convolutional layers and MLP layers, need to unroll matrices or tensors into vectors (and vice versa)
  - called ***vec*** and ***unvec***; use Python `flatten` & `reshape`

Example: detect ones block of length 3

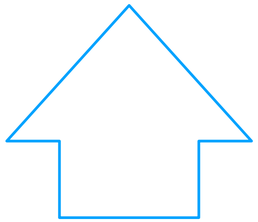
*unlike this example, a real network would **learn** the weights and biases*

$$z_3 = (0 \;\; 0 \;\; 0 \;\; 0 \;\; 1 \;\; 0 \;\; 0 \;\; 0 \;\; 0 \;\; 0)$$



pointwise nonlinearity: $[z - 1]_+$

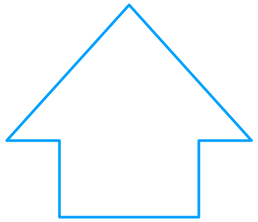
$$z_2 = (0 \;\; -1 \;\; 0 \;\; 0 \;\; 2 \;\; 0 \;\; 0 \;\; -1 \;\; 0 \;\; 0)$$



convolution: weights

$$(1 \;\; 0 \;\; 0 \;\; -1 \;\; 0 \;\; 0 \;\; 0 \;\; 0 \;\; 0 \;\; 0)$$

$$z_1 = (0 \;\; 0 \;\; 0 \;\; 0 \;\; 1 \;\; 0 \;\; 0 \;\; -1 \;\; 0 \;\; 0)$$



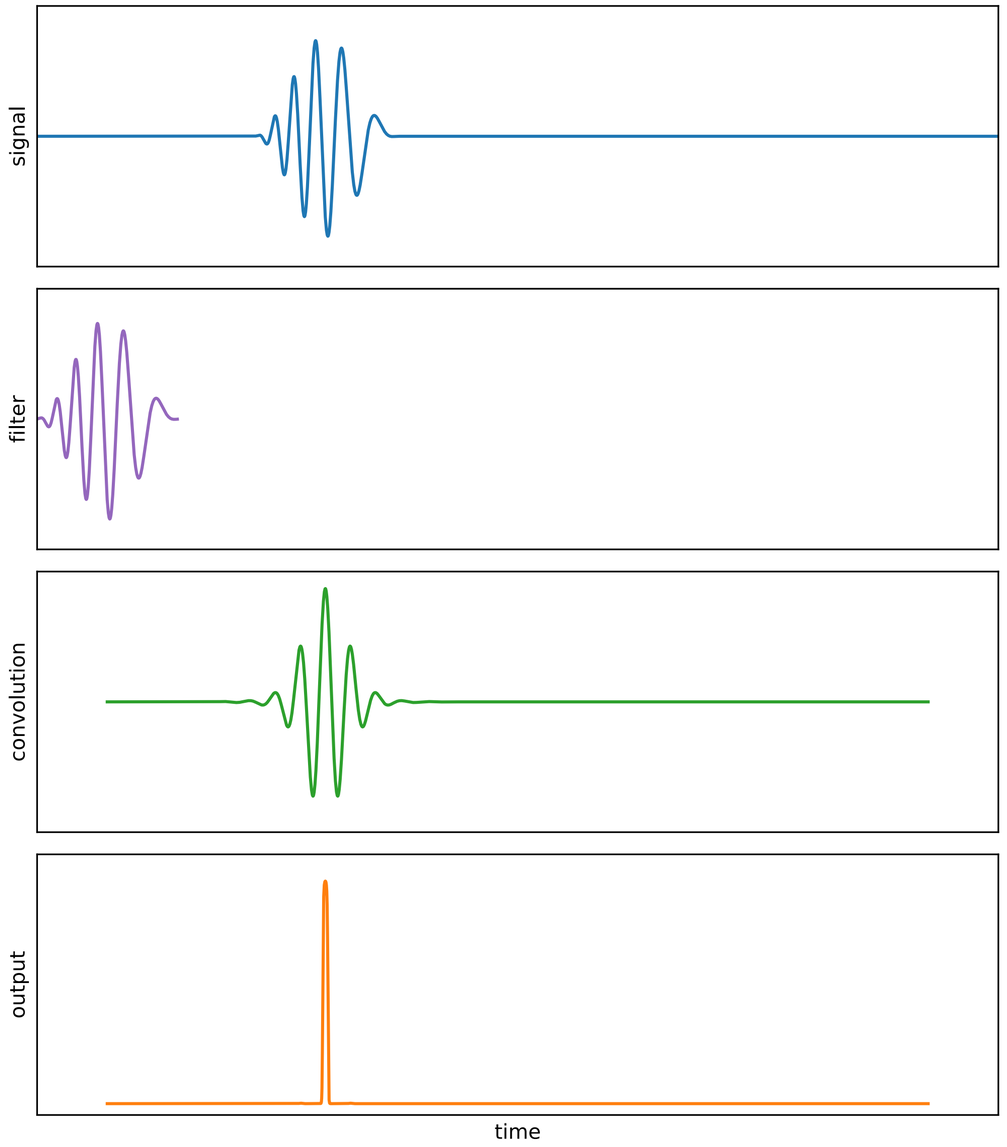
convolution: weights

$$(-1 \;\; 1 \;\; 0 \;\; 0 \;\; 0 \;\; 0 \;\; 0 \;\; 0 \;\; 0 \;\; 0)$$

$$x = (0 \;\; 0 \;\; 0 \;\; 0 \;\; 0 \;\; 1 \;\; 1 \;\; 1 \;\; 0 \;\; 0)$$

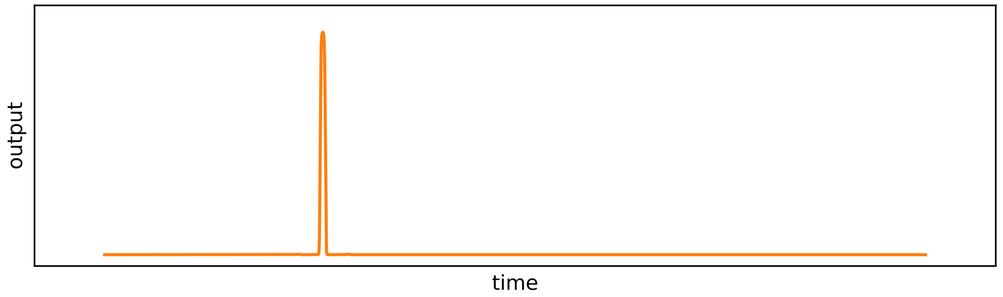
could group these two

Bigger version



- Same idea: detect a chirp in an audio signal
- Convolve input signal w/ chirp filter, then apply sigmoid pointwise

Pooling



$f(t)$

max-pool: $\max_t f(t)$

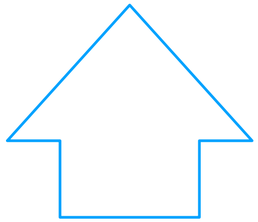
sum-pool: $\sum_t f(t)$

- Convolution gives us translation *equivariance*
- If we want *invariance*,
  - e.g., find the average value of the signal
  - e.g., check whether the max of the signal is ≥ threshold
- Need another step — called *pooling*
  - e.g., max-pool a group of input units = make a single unit whose value is max of inputs

Pooling in action

Change from ***localize*** *to* ***detect***

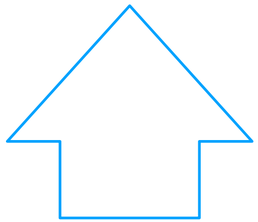
$$z_4 = (1)$$



Max pool: $\max(0,0,0,0,1,0,0,0,0,0) = 1$

Sum pool: $0+0+0+0+1+0+0+0+0+0 = 1$

$$z_3 = (0 \quad 0 \quad 0 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0)$$



Convolution (w/ bias) followed by $[\,\cdot\,]_+$

$$x = (0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 1 \quad 1 \quad 1 \quad 0 \quad 0)$$

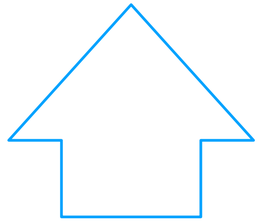
Local pooling

- Might want only *local* invariance (small shifts don't change output much)
- Can do *local* pooling — e.g., average or max of a block of adjacent values
- Can slide a window along signal as in convolution
- Or divide into blocks and pool each block (downsample)

Local pooling example: flexible-length ones block detector

from here, ReLU and pooling can detect or localize

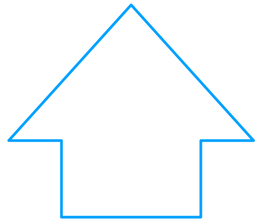
$$z_3 = (0 \;\; -1 \;\; 2 \;\; -1 \;\; 0)$$



convolution: weights

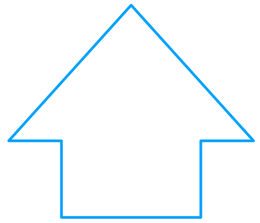
$$(1 \;\; -1)$$

$$z_2 = (0 \;\; 0 \;\; 1 \;\; -1 \;\; 0)$$



sum pool in blocks of 2

$$z_1 = (0 \;\; 0 \;\; 0 \;\; 0 \;\; 1 \;\; 0 \;\; 0 \;\; -1 \;\; 0 \;\; 0)$$



convolution: weights

$$(-1 \;\; 1)$$

$$x = (0 \;\; 0 \;\; 0 \;\; 0 \;\; 0 \;\; 1 \;\; 1 \;\; 1 \;\; 0 \;\; 0)$$

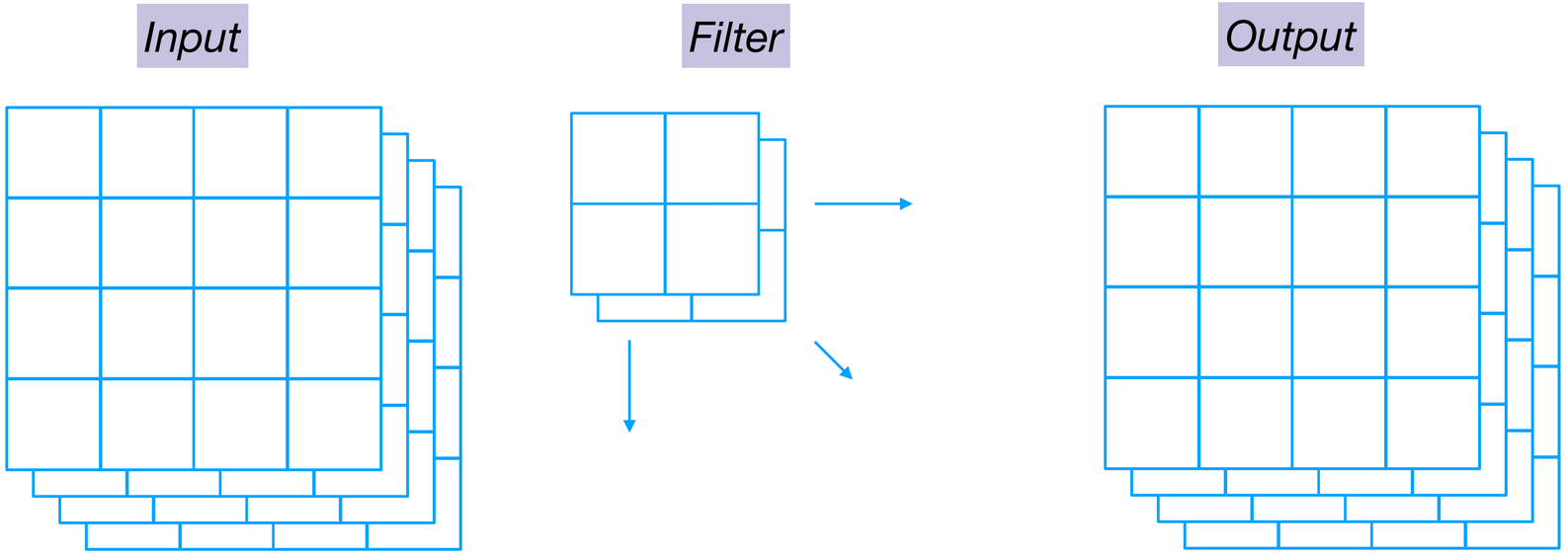
Channels

e.g., $a \in \mathbb{R}^{640\times480}, b_i \in \mathbb{R}^{3\times3}, c_i \in \mathbb{R}$

- Can do multiple convolutions in parallel
  - $a \to (a * b_1 + c_1,\ a * b_2 + c_2, \ldots)$
  - each parameter set is a *head,* each output is a *channel*
  - different heads extract different kinds of info from signal
- Changes shape of activation tensor
  - e.g., if layer has 3d input: multiple heads → 4d output
  - e.g., image → (stack of band-pass images)
- If we think of a tensor as containing multiple channels, the channel index is also called *depth* or *layer* index

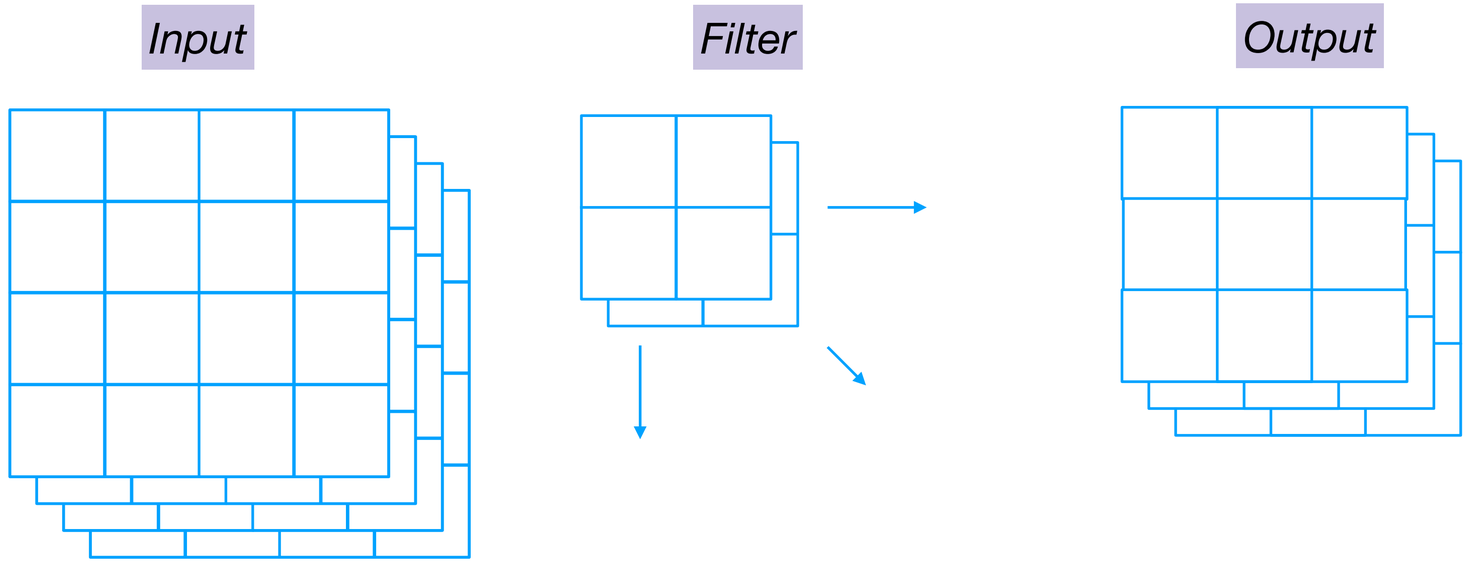
Standard convolution

parameters like dilation and stride can be per-mode



- Without channels, the only option is a ***standard*** convolution: filter has same number of modes as input
  - iterate over all positions in the input, placing the filter at each one and computing dot product
  - in this example, we're choosing the output to be the same size as the input (using padding or cyclic boundary conditions so that we don't have to skip filter placements near edges of input tensor)

Standard convolution (no padding)

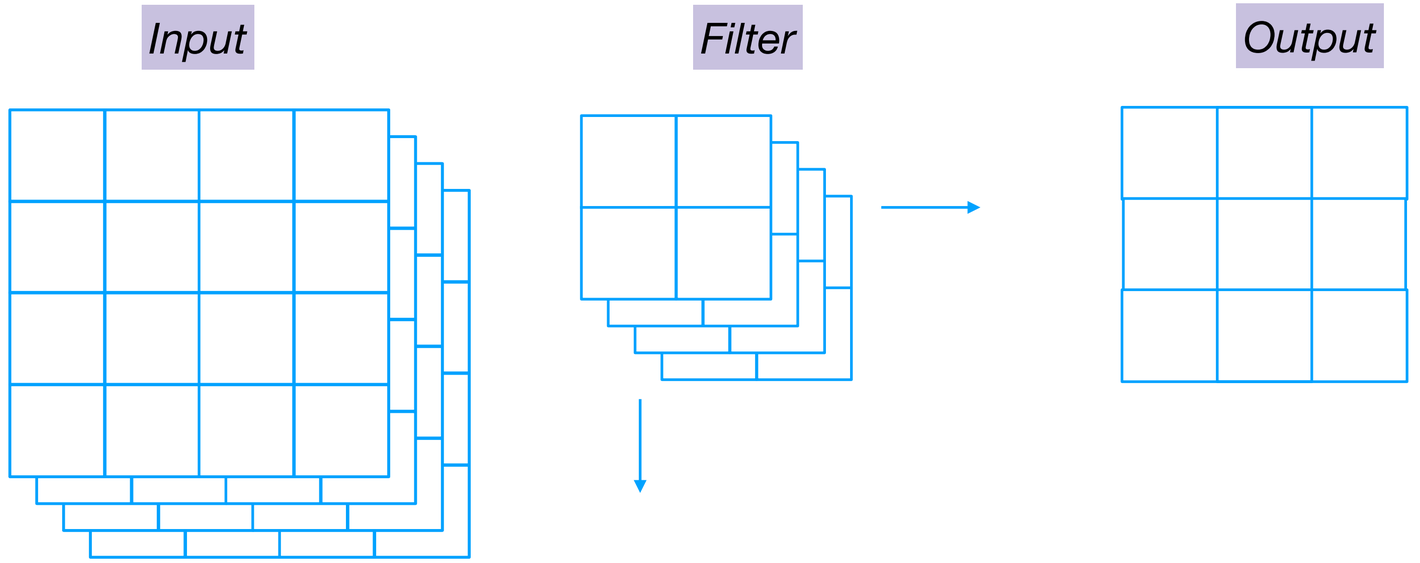


- If we use only ***valid*** positions (no padding or cyclic boundary), output is smaller than input
- Special case: if

```
filter.shape[-1] == input.shape[-1]
```

  - then last mode of output only has length 1
  - if desired we can drop this mode, reducing the number of modes of the output

Standard convolution (no padding)

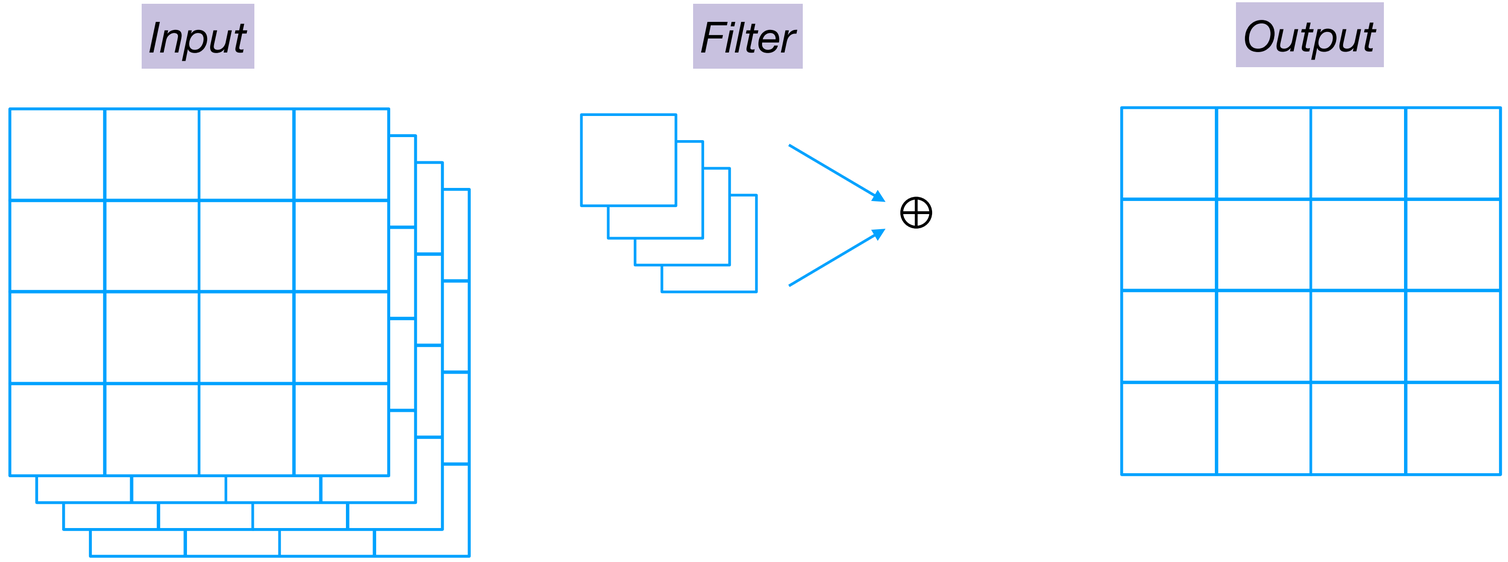


- If we use only ***valid*** positions (no padding or cyclic boundary), output is smaller than input
- Special case: if

```
filter.shape[-1] == input.shape[-1]
```

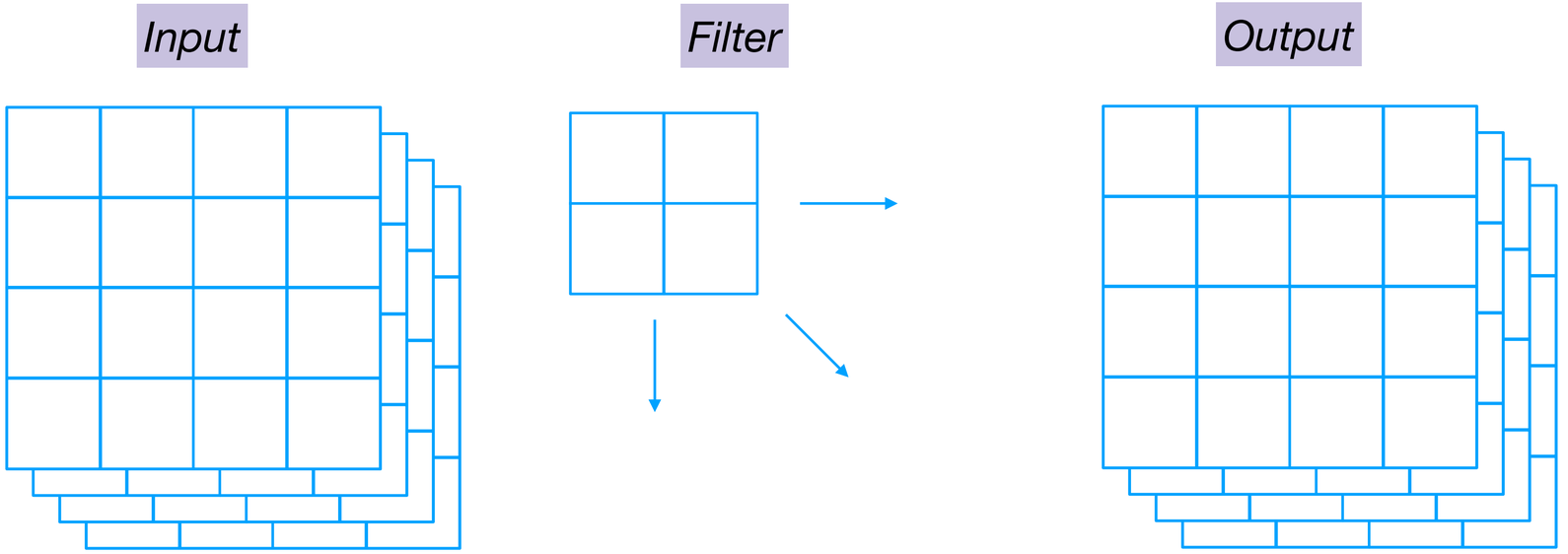
  - then last mode of output only has length 1
  - if desired we can drop this mode, reducing the number of modes of the output

Pointwise convolution



- Even specialer case: filter is $1 \times 1 \times \ldots \times 1 \times$ depth
  - keep all but one mode the same shape, lose last mode
- Called ***pointwise*** or (for 2d) 1×1 convolution
  - same as a weighted sum of slices / channels
- Makes a lot of sense if last mode is channel index: e.g., collapse RGB channels to get overall image intensity
  - common design: multi-head convolution adds a channel index, pointwise later removes it

Same filter on each channel

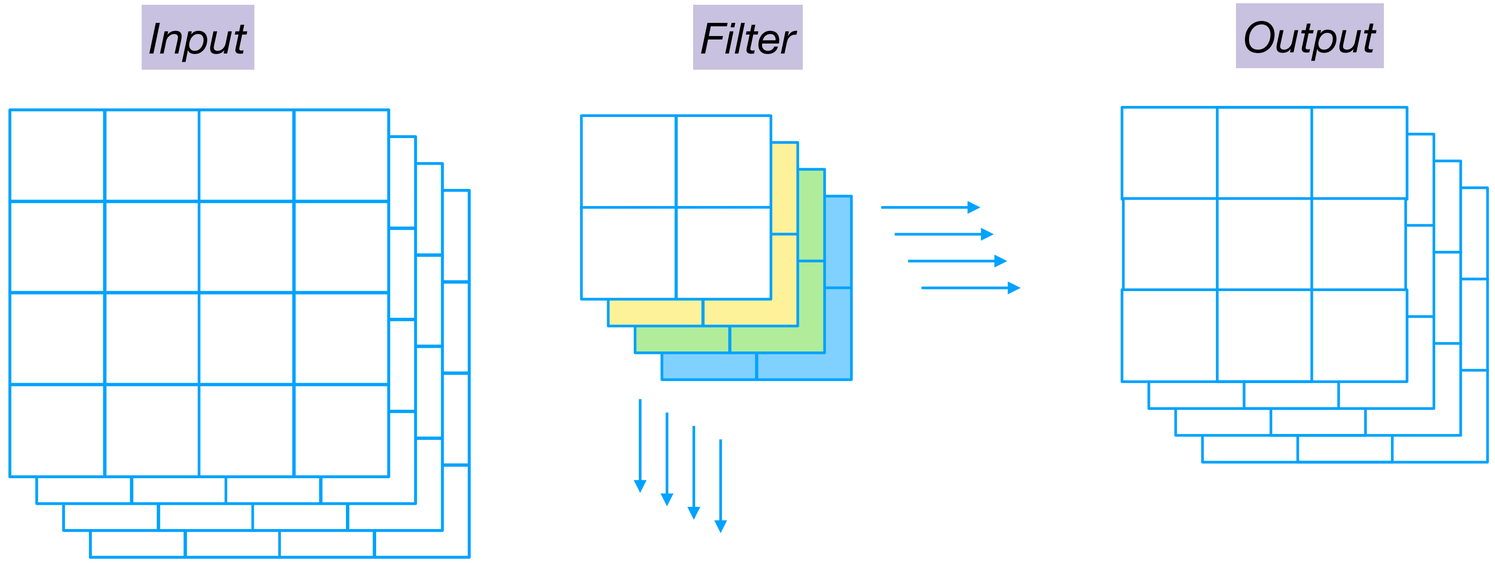


- If last mode is channel index, and if filter has just 1 channel, then standard convolution applies the same filter independently to each channel
  - e.g., blur RGB channels independently by applying a filter like

$$\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$

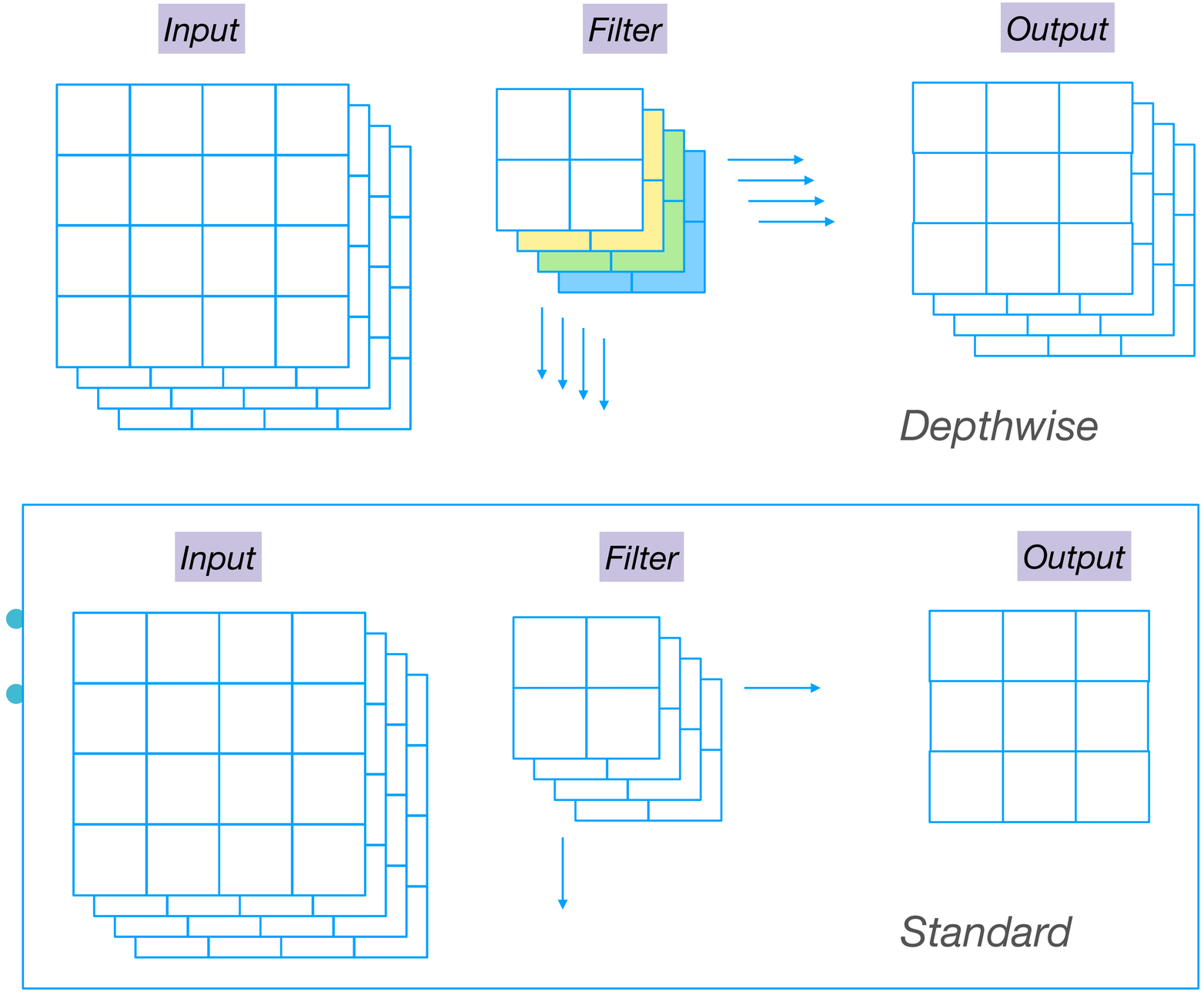
    to each one

Depthwise convolution



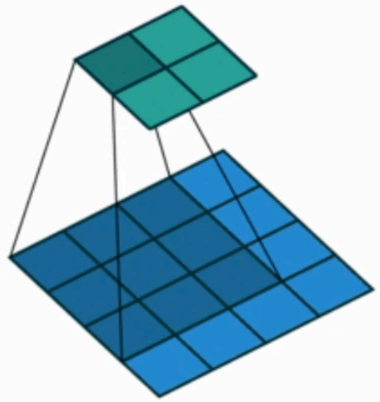
- Might also want to filter each channel separately
- ***Depthwise**:* filter has same number of channels as input, apply to each layer independently
  - this is *different* from a standard convolution: standard = depthwise followed by sum

Depthwise convolution

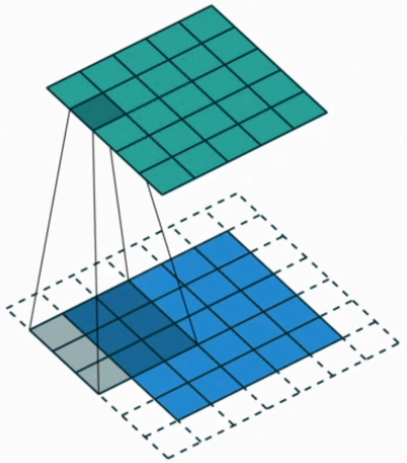


Visualizations

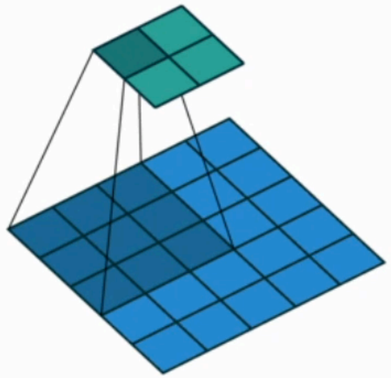
padding = 0, stride = 1



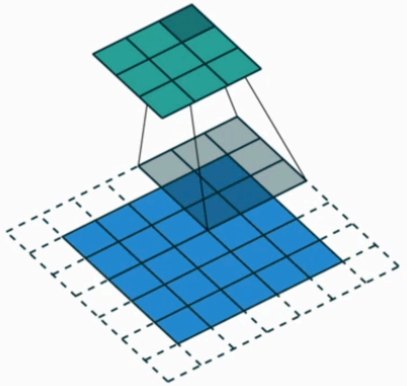
padding = 1, stride = 1



padding = 0, stride = 2

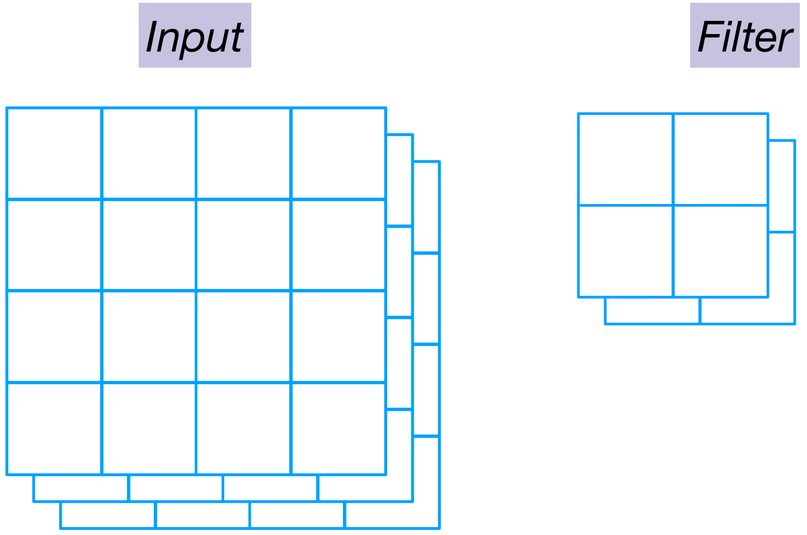


padding = 1, stride = 2



link to animations

Calculating sizes



- With all the options, it can be tricky to figure out what size the output of a convolution will be
  - e.g., given $4 \times 4 \times 3$ input and $2 \times 2 \times 2$ filter
  - what size is output for a standard, valid convolution?
  - what if we zero-pad the last (depth) mode by 1 at end?
  - what if we use stride 2, 2, 1 (with no padding)?

3 x 3 x 2

3 x 3 x 3

2 x 2 x (2 or 3)

Try it

- Suppose our input is 13×13×5, our filter is 3×3×5, and we compute only valid output positions. Suppose we zero-pad by 1 on all sides of the images (i.e., in the first 2 modes) and use a stride of 2 in all modes. What shape is the output?

  A. 6×6×1

  B. 7×7×1

  C. 13×13×5

  D. 13×13×1

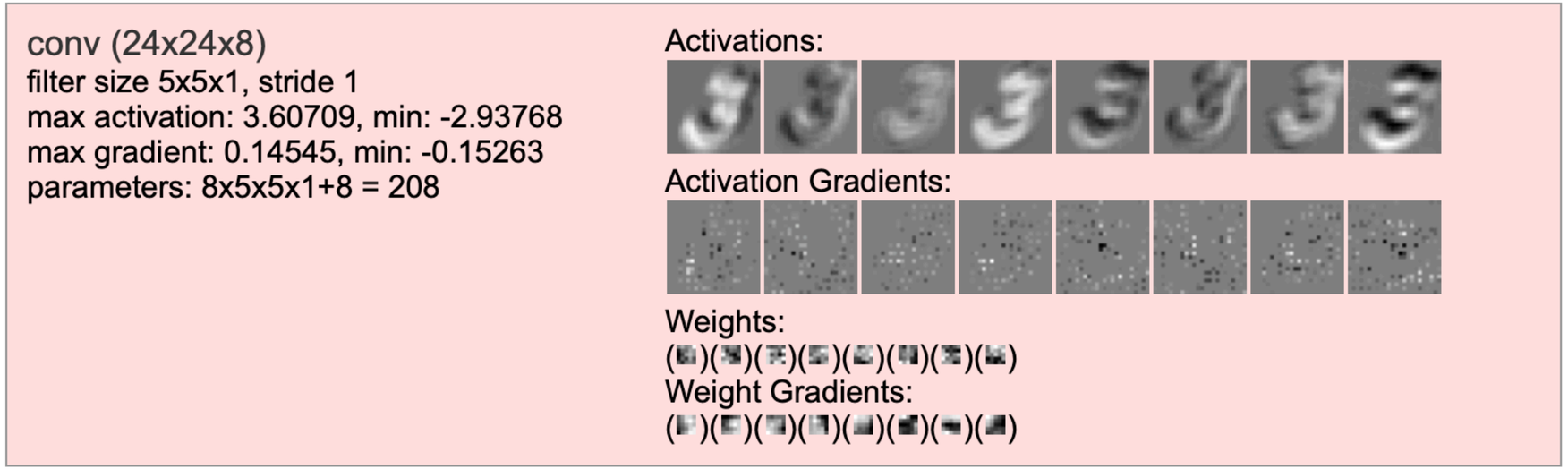
  E. 7×7×5

Putting it together

- Convolution and pooling work well together: tunable between equivariance and invariance
- Often do convolutions alternating with nonlinearities, occasional pooling to reduce size of a layer, occasional multiple heads or pointwise convolutions to change number of modes of tensor
- Sometimes increase number of channels as we downsample (else we force the net to discard a lot of information)
- Can use vec or unvec to glue between convolutional layers and plain MLP layers
- Common pattern: more smaller channels as we go deeper (using multi-head convolution, subsampling by pool or stride)
- Add in residual connections to help gradients not explode or vanish

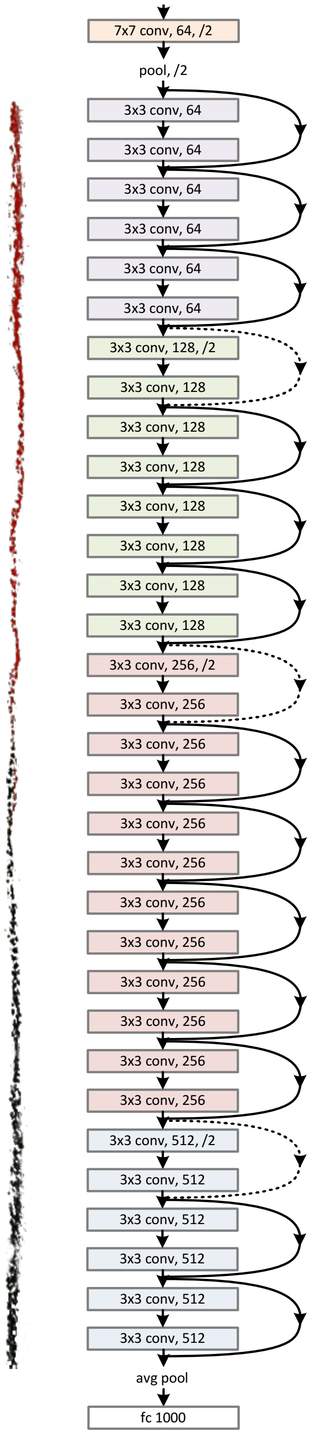
Interactive example

https://cs.stanford.edu/people/karpathy/convnetjs/demo/mnist.html

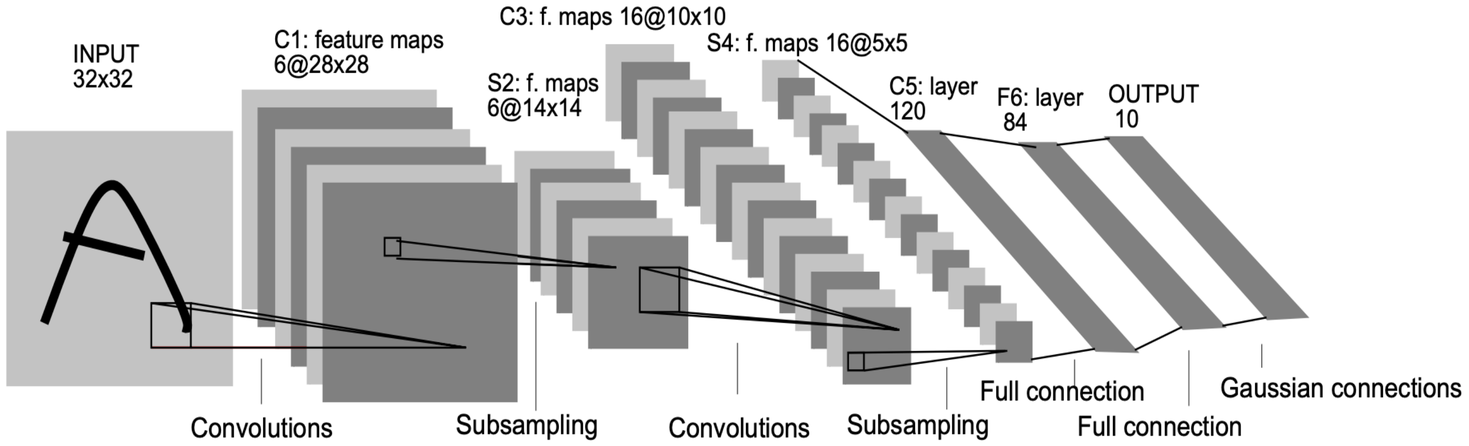


- CNN for MNIST, trained by Javascript in browser
- 2 × (convolution + ReLU + max-pool) + FC + softmax
- Enough for 99% validation accuracy

Lots of well-known convolutional networks



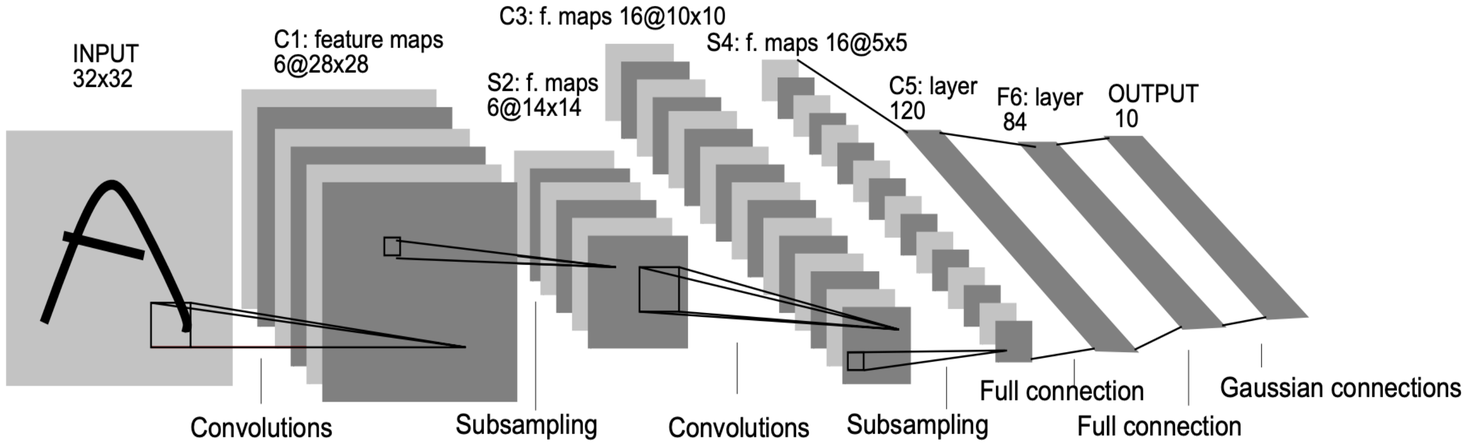
[He et al., 2015]



[LeCun et al., 1998]

- These features have appeared in *many* famous models (SOTA at their time)
  - e.g., LeNet (convolution and pooling, for MNIST digit classification)
  - e.g., ResNet (convolution, pooling, batch norm, residual connections, for ImageNet)

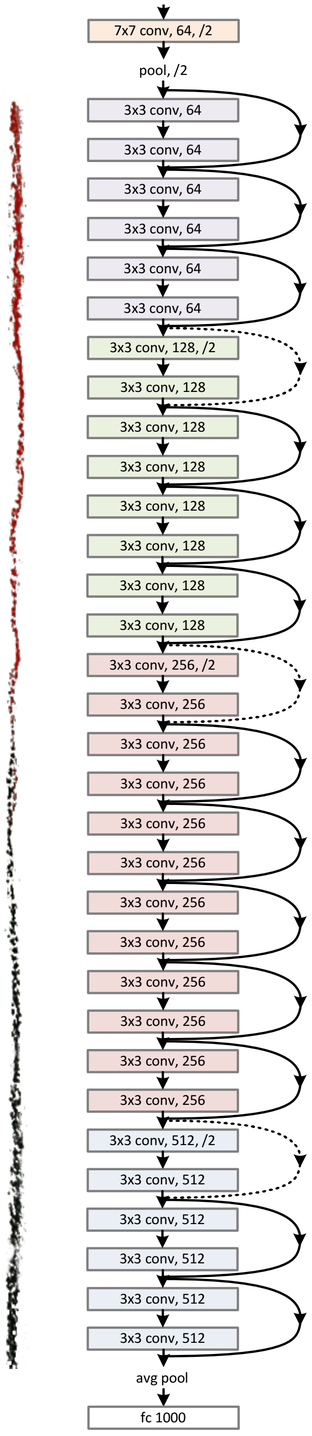
LeNet



[LeCun et al., 1998]

- Convolutions are 5×5 with no padding
- Subsampling is average-pooling followed by a pointwise linear function and nonlinearity
- Nonlinearities are all tanh, symmetric version of sigmoid
- C3 uses a hand-designed sparse connection pattern
- After S4, we vec the 5×5×16 tensor, use MLP layers
- Final nonlinearity is not commonly used any more: it unvecs the F6 activation to a 12×7 image and compares to 10 digit templates using Gaussian RBFs

ResNet-34



- 3×3 conv, 128, /2 means a 3×3 convolution with stride 2, 128 filters
- At the top is a max-pool, and at the bottom we pool by averaging each channel to a single number
- Residual connections throughout
  - dashed ones incorporate downsampling of the identity path via a 1×1 convolution of stride 2

[He et al., 2015]

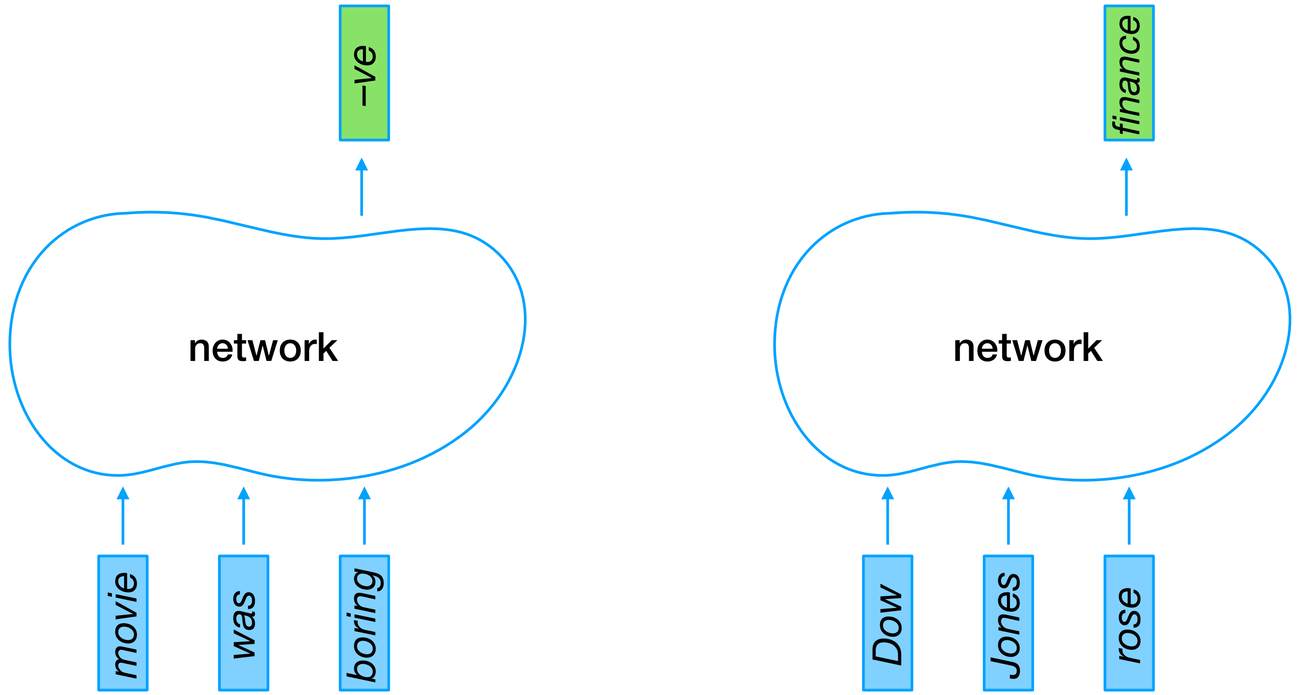
Learning objectives—CNNs

- What is a (continuous time or space) signal? How does it relate to a (discrete time or space) sampled vector, matrix, or tensor?
- What are important properties of convolutional layers? (invariance/equivariance, few parameters, fast forward/backward pass)
- Explain variations: output size, padding, dilation, stride
- What kinds of filters can convolution+nonlinearity represent? (blur, sharpen, edge/corner detection, band-pass, …)
- What kinds of operations control layer shape and size? (pooling, vec/unvec, multiple heads, 1×1 convolution)
- Explain tensor indexing, broadcasting, elementwise notation, Einstein summation, representation of derivatives
- Carry out (small) tensor operations and convolutions by hand, including predicting the shape of output tensor

Recurrent networks

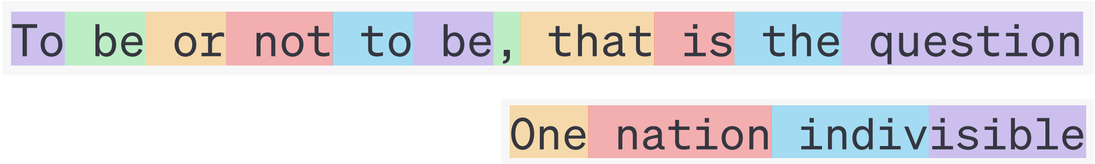
10-701 Introduction to Machine Learning
Geoff Gordon

Sequence prediction



- Models up to now have *fixed-shape* inputs
- Many language tasks (like sentiment classification, topic classification) take *variable-length* sequence as input
- Need models that can scale to fit

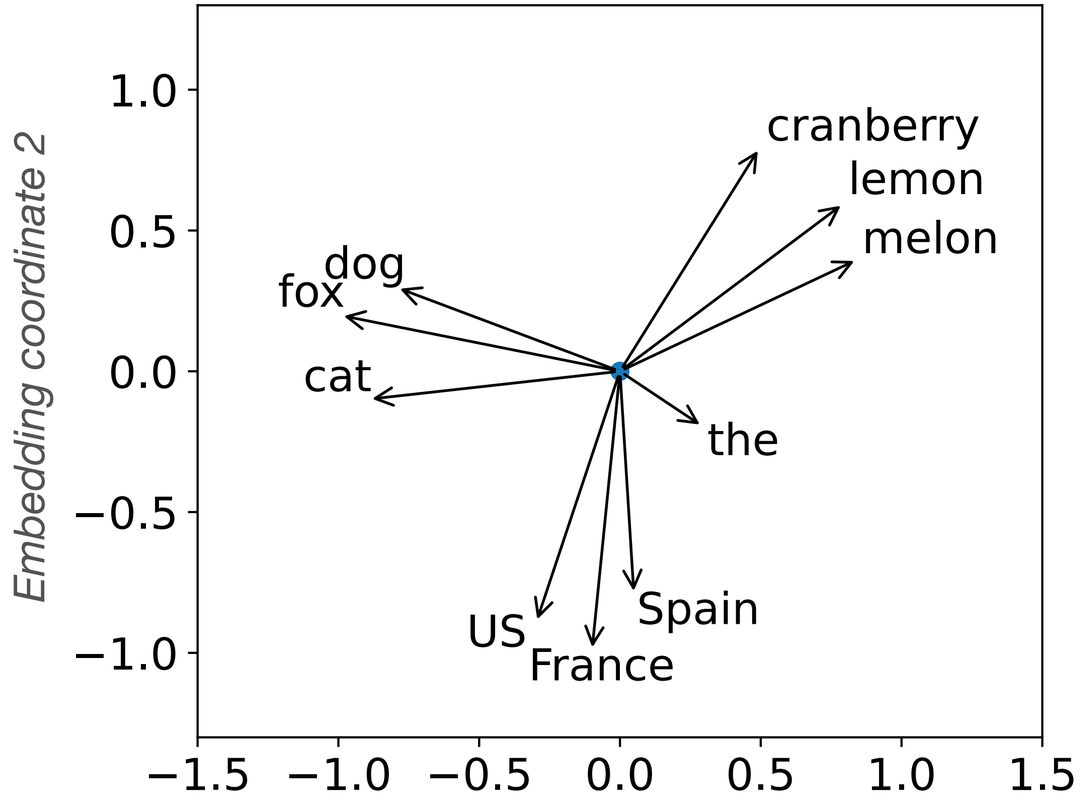
Discrete-valued sequences



credit: OpenAI tokenizer

- Another way language differs: ***discrete*** tokens instead of continuous pixel values
  - characters
  - words
  - word parts
  - class labels — potentially 1000s with relationships among them

Embedding

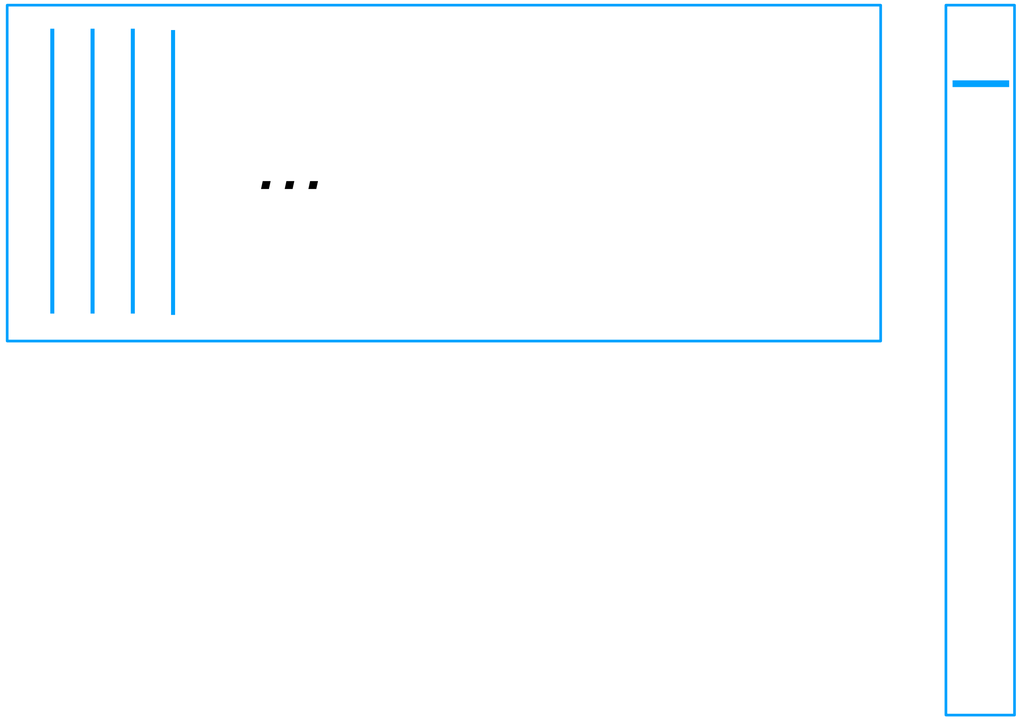


- Handle tokens by *embedding*: replace by vector $\in \mathbb{R}^d$ (d might be 512, 2048, 10k — large but $\ll$ vocabulary size)
  - ideally: relationships among tokens reflected in vectors
  - dog and cat embedded near each other
  - Paris – France + Spain $\approx$ Madrid

Learning embeddings

- If each raw token = one-hot vector, and each embedded token = arbitrary vector, this is a ***linear map***
- $\mathbb{R}^{\text{vocabulary size}} \to \mathbb{R}^{\text{embedding dimension}}$
- Embedding of "dog" = Ve_{dog}

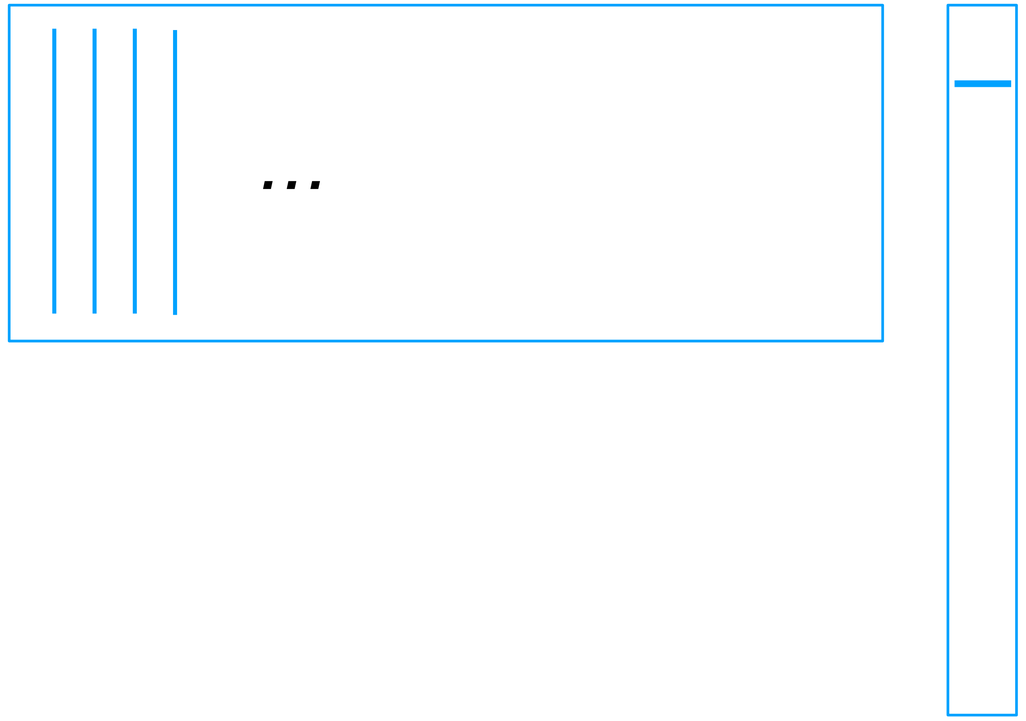
$$Ve_{\text{dog}} =$$



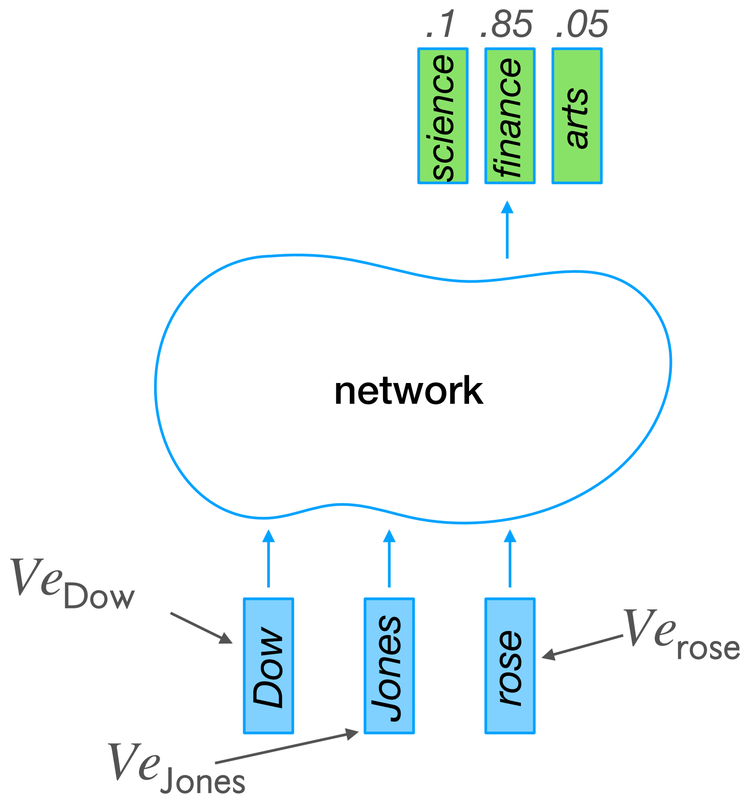
Learning embeddings

- If we present tokens as one-hot (for a neural net model in any task), first layer automatically learns embedding $V = W_1$
- SGD step updates embeddings of current tokens to make current prediction more accurate
- We'll talk later about more ways to learn embeddings

$$V e_{\text{dog}} =$$

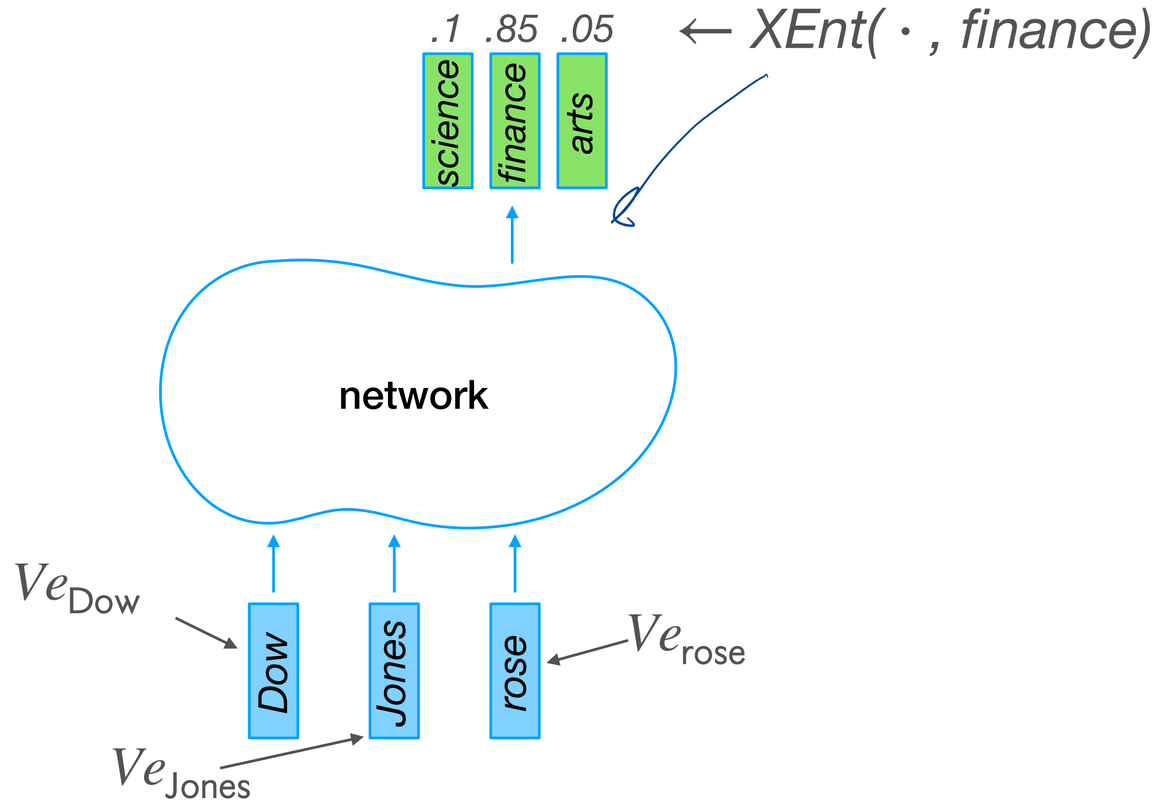


Sequences of tokens



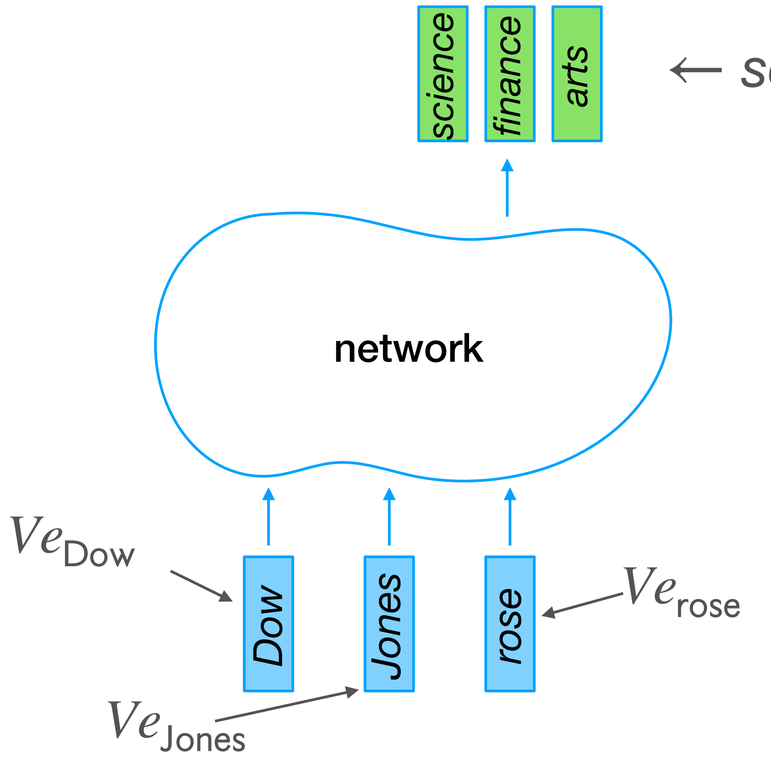
- Putting it together:
  - first encode the input as a sequence of tokens
  - then embed each token
  - then pass tokens to the model one by one
  - finally read off predictions at output node

Train



- Apply loss to prediction (maybe target was *finance*)
- Backprop through network
- Use gradient to update parameters (SGD, Adam, …)
  - including embeddings of tokens that appeared in input

Class label embeddings



= softmax of

$z \cdot W^\top e_{\text{lbl_science}}$

$z \cdot W^\top e_{\text{lbl_finance}}$

$z \cdot W^\top e_{\text{lbl_arts}}$

...

- Can even interpret class labels as tokens with embeddings
  - if penultimate layer activations are z
  - and last layer is softmax(Wz)
  - then we can think of ith row of W as embedding of ith label
  - and we put highest probability on the label whose embedding has highest dot product with z